



ZiloCart

ApBot E-Commerce Assistant

SRS Requirements, Chatbot Design & Technical Documentation

THEME

E-Commerce Assistant — Unleashing the Power of AI/ML

PROJECT TEAM

Mohammad Hamza

Ibrahim Bawany

Usman Bawany

Yousuf

DOCUMENT VERSION

Version 1.0 • August 2026

Document Control & Project Team

Document information

Item	Details
Project	ApBot — AI/ML E-Commerce Assistant
Host application	ZiloCart website
Primary subject	ApBot chatbot requirements, design, training, testing and integration
Reference	Supplied ApBot Software Requirements Specification
Version	2.0 — ApBot-focused revision
Date	August 2026

Project team

#	Name	Role
01	Mohammad Hamza	Project team member
02	Ibrahim Bawany	Project team member
03	Usman Bawany	Project team member
04	Yousuf	Project team member

Document boundary

This report is limited to ApBot and the requirements stated in the supplied SRS. ZiloCart is discussed only where the SRS requires a host website, a chat frontend, REST integration or checkout navigation. Store-wide administration, database design and unrelated website modules are intentionally outside this document.

Intended audience

The document is prepared for project evaluators, developers and stakeholders who need to understand what ApBot does, how it is trained, how a message is processed and how the chatbot is connected to the host website.

Table of Contents

Section	Title	Page
1	Background, Necessity & Proposed Solution	4
2	Purpose, Scope, Constraints & Assumptions	5
3	SRS Functional Requirements	6
4	Non-Functional & Interface Requirements	7
5	ApBot System Architecture	8
6	DFD Level 0 — ApBot Context	9
7	DFD Level 1 — ApBot Processes	10
8	Intent Dataset & Pattern Matching	11
9	NLU, Entities, Context & Expectations	12
10	NLP, Tokenisation & Bag of Words	13
11	TensorFlow/Keras Model & Training	14
12	Conversation & Response Design	15
13	Frontend, Backend & REST Integration	16
14	ApBot Testing & Acceptance	17
15	Installation & SRS Deliverables	18
16	ApBot Screenshot Placeholders	19
17	Technical Evidence & Conclusion	20

Key terminology

Term	Meaning
ApBot	Intent-classification e-commerce assistant defined by the SRS.
Intent	The purpose of a message, such as greeting, product search or checkout.
NLU	Identification of user intent, entities, context and expected outcome.
NLP	Preparation of human language for computational processing.
BoW	Bag of Words; a numerical vector showing which vocabulary terms are present.

Background, Necessity & Proposed Solution

Background and necessity

E-commerce users often need immediate answers while browsing. Traditional support channels can create waiting time, operate only during fixed hours and repeatedly handle the same questions. A chatbot reduces this delay by remaining available throughout the website visit and by answering routine product, order, payment, delivery and return questions through natural conversation.

The SRS identifies additional benefits: 24/7 availability, quick responses, scalability, reduced pressure on human support and a more personalised customer experience. An e-commerce assistant can guide a customer at the exact point where help is needed instead of requiring a phone call or email.

Problem definition

- Customers may not know the correct product name or search terms.
- The same product, delivery, payment and order questions are asked repeatedly.
- Static FAQs cannot interpret varied wording, slang or incomplete questions.
- Customers need a clear next step when they decide to purchase a product.
- The assistant must return useful guidance without making unrelated or misleading claims.

Proposed ApBot solution

ApBot is a text-based AI/ML assistant trained on conversational patterns. NLTK prepares the message, Bag of Words converts it to numeric input and a TensorFlow/Keras classifier predicts the most likely intent. The backend selects an appropriate response from intents.json and returns it to the website through a REST endpoint.

Main objectives

- Welcome and guide the customer naturally.
- Explain products, offers and suitable alternatives.
- Answer order, checkout, payment, delivery, return and support questions.
- Help the customer decide on a product and continue to checkout.
- Provide a safe, professional fallback for questions outside ApBot's scope.

Purpose, Scope, Constraints & Assumptions

Purpose of this document

The purpose of this document is to describe ApBot's requirements, architecture, language-processing pipeline, model training process, conversation design, REST integration and verification approach. It explains the constraints under which the chatbot operates and provides traceability from the supplied SRS to the implemented design.

ApBot scope

- Accept regular human text through a browser-based chat interface.
- Match user messages to trained intent categories and scripted response groups.
- Provide product information, available-offer guidance and similar alternatives.
- Assist with frequently asked questions, order tracking and common customer issues.
- Guide a customer toward a suitable product and provide checkout navigation.
- Offer contact-support guidance when a human representative is needed.
- Expose chatbot functionality to the host website through a JSON REST API.

Constraints

- Prediction quality depends on the coverage and quality of training patterns.
- Spelling mistakes, slang, mixed topics and ambiguous requests can reduce confidence.
- ApBot is an intent classifier, not an unrestricted general-knowledge assistant.
- Responses are limited to defined intents and approved response content.
- The chatbot requires its vocabulary, class labels and trained model artifacts at runtime.

Assumptions and exclusions

Users have a modern browser and a stable connection to the host application. Product and checkout actions may be supplied by the website integration, but the internal design of the complete store is not part of this ApBot SRS report. ApBot does not guarantee correct understanding of every possible human sentence and must fail safely when a message is unsupported.

SRS Functional Requirements

The following matrix maps the functional statements in the supplied SRS to the ApBot implementation and design evidence.

SRS requirement	ApBot implementation evidence	Coverage
Welcome user	Natural greeting and help prompt in the chat interface.	Covered
Products and offers	Product-search and offers intents return relevant guidance.	Covered
Product details	Product-details intent explains selected product information.	Covered
Similar alternatives	Similar-products intent supports alternative-brand requests.	Covered
Contact representative	Contact-support intent provides escalation guidance.	Covered
Pattern matching	intents.json stores anticipated patterns, tags and responses.	Covered
Conversation context	Backend may use the active request/session context when selecting actions.	Covered
NLU	Intent, entity, context and expected outcome are identified.	Covered
NLP	Tokenisation, normalisation and stemming prepare user text.	Covered
Word vectorisation	Binary Bag of Words converts known vocabulary into numbers.	Covered
Keras/TensorFlow	Dense neural network classifies the input into one of 21 intents.	Covered
Human conversation	The input is ordinary text rather than a rigid command syntax.	Covered
Best-product help	Recommendation intent provides decision-support responses.	Covered
Checkout redirect	Checkout intent may return a website navigation action.	Covered
Frontend	Floating web chat captures input and displays the reply.	Covered
Backend and REST	POST /api/chat validates input and returns JSON.	Covered
ML model output	Prediction returns an intent and probability for backend use.	Covered

Non-Functional & Interface Requirements

Non-functional requirements

Quality	ApBot design response
Compatibility	Standards-based HTML, CSS and JavaScript support current desktop and mobile browsers.
Security	The endpoint validates JSON, limits message length and relies on host access controls for protected actions.
User experience	Clear messages, typing feedback, quick actions and graceful fallback keep conversation understandable.
Performance	The model is loaded once; preprocessing and local prediction are lightweight for short messages.
Reliability	Missing input and unsupported topics return controlled responses rather than application errors.

Hardware interface requirements

Item	Minimum stated by SRS
Processor	Intel Core i5 or higher
Memory	8 GB RAM or higher
Storage	500 GB hard-disk space
Display	Colour SVGA or equivalent
Input	Mouse and keyboard

Software interface requirements

Area	Technology
Frontend	HTML5, CSS and JavaScript chat interface
Data store	JSON intent dataset and local model artifact files
Programming	Python 3.x and Jupyter Notebook / JupyterLab
AI/ML libraries	TensorFlow, Keras and NLTK
Integration	Flask REST endpoint using JSON request and response bodies

ApBot System Architecture

The SRS defines three primary components: frontend, backend and machine-learning model. The architecture below keeps those responsibilities separate and shows only the files and data flows required by ApBot.

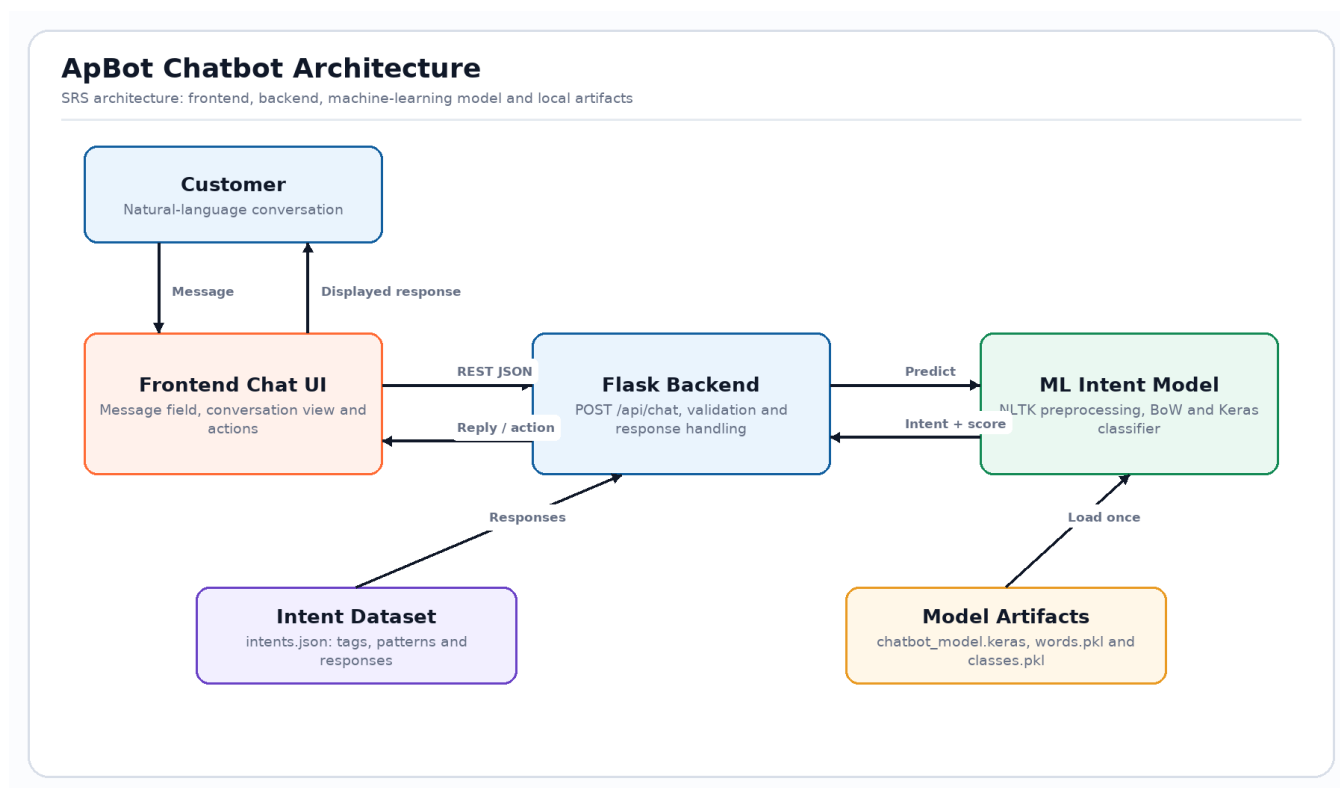


Figure 1 — ApBot frontend, backend, model and artifact architecture

Component responsibilities

Component	Responsibility
Frontend chat UI	Capture a message, show conversation state, render the reply and perform an approved website action.
Flask backend	Validate the request, call the engine, select response data and return a stable JSON contract.
NLTK / BoW pipeline	Normalise user text and create the numeric vector expected by the trained model.
Keras model	Produce probability scores and identify the most likely intent class.
Intent and model files	Preserve patterns, responses, vocabulary, class order and trained weights.

DFD Level 0 — ApBot Context

At context level, ApBot is treated as one system. The customer provides a natural-language request, the host website carries the REST message, and ApBot uses its intent dataset and trained artifacts to return an answer or a permitted navigation action.

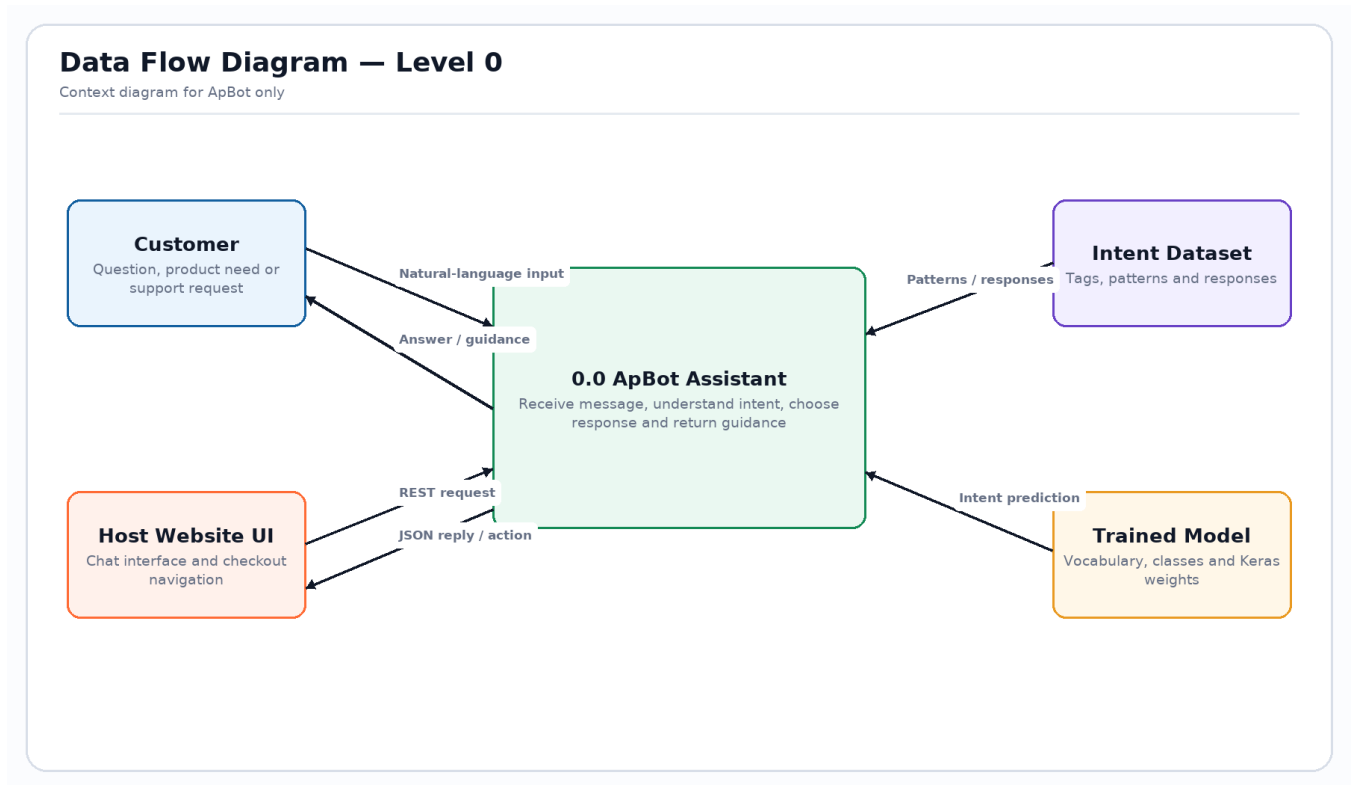


Figure 2 — ApBot Level 0 context diagram

External data flows

- Customer → ApBot: greeting, question, product need, order query or support request.
- ApBot → customer: natural response, clarification, guidance or checkout direction.
- Website UI → ApBot: validated JSON request containing the message.
- ApBot → website UI: JSON reply containing result fields and an optional action.
- Intent/model files → ApBot: response groups, vocabulary, classes and learned weights.

DFD Level 1 — ApBot Processes

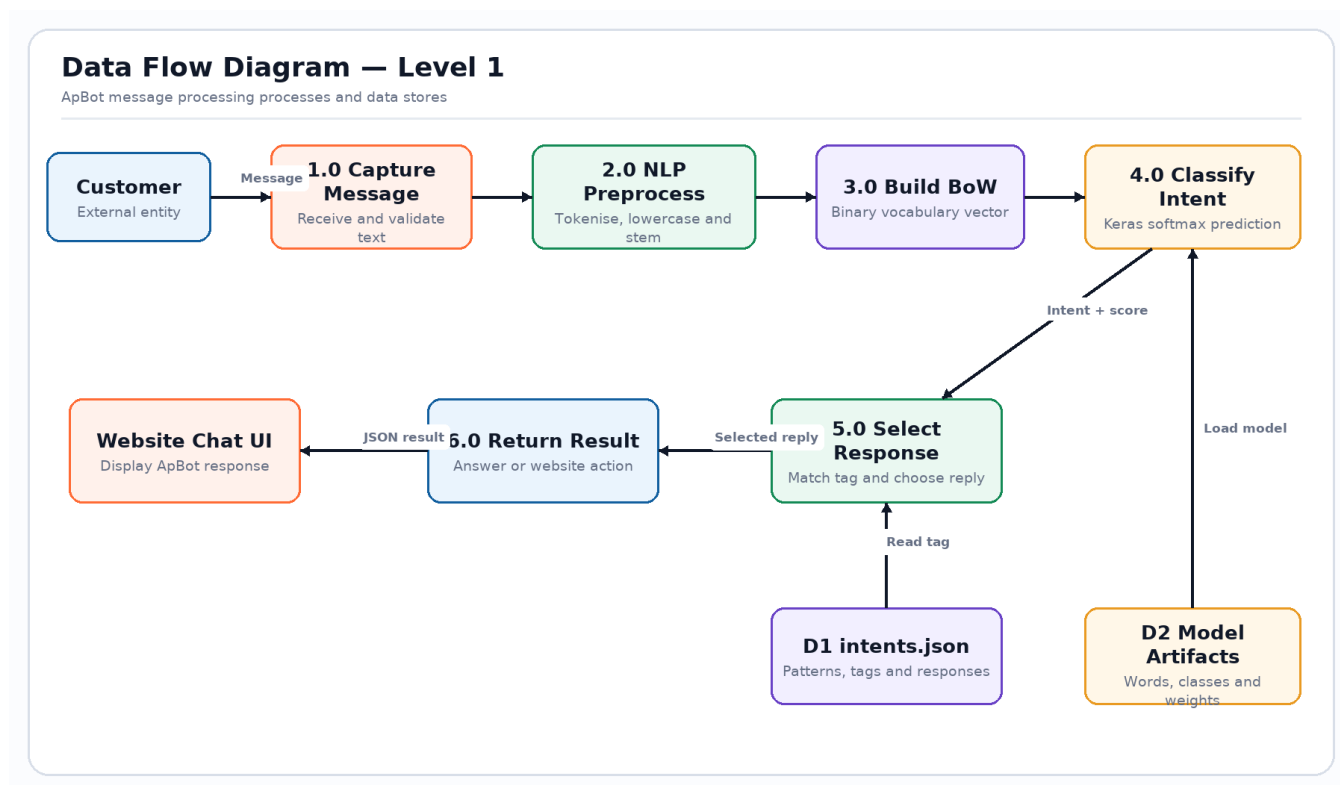


Figure 3 — ApBot Level 1 processing diagram

Process summary

Process	Description
1.0 Capture Message	Receive text and reject missing or invalid input.
2.0 NLP Preprocess	Tokenise, lowercase, filter noise and stem tokens.
3.0 Build BoW	Map the processed message to the saved vocabulary order.
4.0 Classify Intent	Run Keras inference and obtain class probabilities.
5.0 Select Response	Match the predicted tag to intents.json and choose a reply.
6.0 Return Result	Send the answer and optional website action to the frontend.

Intent Dataset & Pattern Matching

Dataset structure

ApBot uses chatbot/intents.json as its conversational training datastore. The current dataset contains 21 intent classes and 1,065 patterns. Each intent is a small supervised-learning category with examples of how a user may express the same purpose.

JSON field	Purpose	Example
tag	Unique intent label used as the target class.	product_search
patterns	Alternative user messages used for model training.	Show me some laptops
responses	Approved replies associated with the predicted tag.	I can help you find a product.

Implemented intent catalogue

Group	Intent tags
Conversation	greeting, goodbye, thanks, help, unknown
Product assistance	product_search, product_details, similar_products, product_comparison
Decision support	recommendation, offers, wishlist
Purchase journey	cart, checkout, payment
Orders and delivery	order_tracking, cancel_order, shipping
After-sales help	returns, contact_support
Account	account

Pattern-matching principle

Training patterns are not compared as exact sentences. They teach the classifier which vocabulary combinations are associated with each intent. This allows messages with different wording to map to the same response category, while the unknown intent provides controlled handling when no shopping-related class is suitable.

NLU, Entities, Context & Expectations

Natural Language Understanding in ApBot

The SRS describes NLU through three ideas: entities, context and expectations. ApBot applies these ideas around intent classification so that a predicted class produces an appropriate e-commerce outcome rather than only a technical label.

NLU element	Meaning	ApBot example
Intent	The overall purpose of the message.	checkout, shipping or product_details
Entity	A meaningful value inside the message.	product term, category, brand or budget
Context	Relevant background available during the active request.	current chat action or signed-in website state
Expectation	The useful result the user expects.	answer, product guidance, support or redirect

Understanding sequence

- Read the complete message rather than requiring a fixed command format.
- Normalise the text and identify known vocabulary signals.
- Predict the most likely intent and compare its confidence with the engine threshold.
- Use the predicted tag to select an approved response group.
- Return a clear result or a professional scope explanation when the message is unsupported.

Context boundary

The classifier itself predicts one message at a time; it is not a generative long-term memory system. Any website state used by the integration is limited to the active customer session. ApBot should never invent context, expose another customer's information or pretend to understand an unsupported request.

Expected conversational qualities

- Relevant to the predicted shopping intent.
- Short enough to read comfortably inside a chat window.
- Professional, courteous and explicit about the next step.
- Safe when confidence is low or the topic is outside the defined domain.

NLP, Tokenisation & Bag of Words

NLP preprocessing pipeline

Natural Language Processing converts the customer's text into a repeatable format that can be consumed by the neural network. The training notebook and runtime engine use the same saved vocabulary order so that each vector position has the same meaning during training and inference.

Stage	Operation	Purpose
1. Tokenise	wordpunct_tokenize splits text into word-like tokens.	Separate the message into processable units.
2. Normalise	Convert tokens to lowercase and remove unsuitable noise.	Reduce differences caused by casing or punctuation.
3. Stem	PorterStemmer reduces related words to a common stem.	Treat products/product-like forms consistently.
4. Vectorise	CountVectorizer creates a binary Bag-of-Words row.	Represent the message numerically.

Bag-of-Words representation

Let the saved vocabulary contain [buy, checkout, laptop, phone, return]. For the message “buy laptop”, the binary vector is [1, 0, 1, 0, 0]. A value of 1 means the vocabulary term is present; 0 means it is absent. Word order is not stored, which keeps inference simple and fast but also limits deep sentence understanding.

Vocabulary term	buy	checkout	laptop	phone	return
Vector value	1	0	1	0	0

Saved preprocessing artifacts

- words.pkl stores the exact input vocabulary and feature order.
- classes.pkl stores the output intent labels in model order.
- The runtime must apply the same tokenisation and stemming rules used during training.

TensorFlow/Keras Model & Training

Neural-network design

Layer / setting	Function
Input vector	One binary feature for every saved vocabulary term.
Dense 64, ReLU	Learn combinations of words that indicate an intent.
Dropout 0.30	Reduce memorisation and improve generalisation.
Dense 32, ReLU	Refine the learned intent representation.
Softmax output	Produce one probability-like score for each of 21 classes.
Loss	Sparse categorical cross-entropy for integer class labels.
Optimiser	Adam optimiser for iterative weight updates.
Early stopping	Monitor validation loss and restore the best model weights.

SRS development sequence

Step	Implementation activity
1	Prepare NLU training data in intents.json.
2	Define tokenisation, stemming, vocabulary and model configuration.
3	Compile and fit the TensorFlow/Keras classifier.
4	Evaluate training/validation behavior and test unseen messages.
5	Save artifacts and integrate the chatbot with the website.

Recorded training result

A representative notebook run recorded approximately 99.41% best training accuracy and 85.92% best validation accuracy. The lower validation score is reported separately because it is a more realistic indication of performance on held-out examples. Accuracy alone does not guarantee correct handling of every sentence, so intent-level conversation tests remain necessary.

Conversation & Response Design

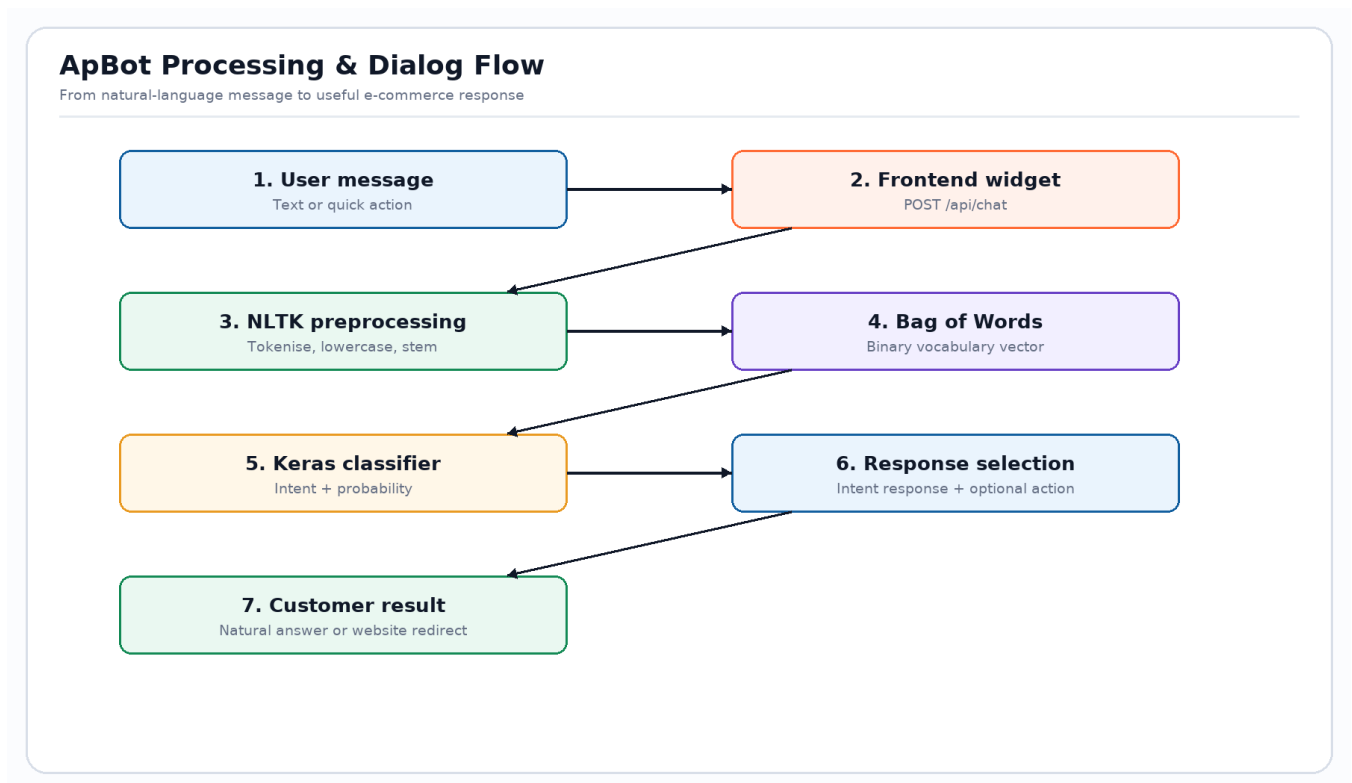


Figure 4 — ApBot message-processing and dialog flow

Response-selection behavior

- The customer enters natural text or chooses a permitted quick action.
- The backend predicts an intent and identifies the corresponding tag.
- A response is selected from the approved responses for that tag.
- The website displays the text and may perform an optional checkout or page action.
- Low-confidence or irrelevant input receives a professional scope response.

Representative ApBot dialog

Speaker	Message
ApBot	Hi, I'm ApBot. How can I help you today?
Customer	I need help choosing a laptop.
ApBot	I can help. Tell me your preferred use or budget.
Customer	How can I continue with my purchase?
ApBot	You can review your selection and continue to checkout.

Frontend, Backend & REST Integration

The three SRS components

Component	Implemented ApBot responsibility
Frontend	Provides the chat launcher, message input, conversation history, typing state and visible assistant reply.
Backend	Receives the message, validates it, calls the engine and creates the JSON response.
ML model	Transforms the prepared message into scores and identifies the most likely intent.

REST endpoint

The website integrates ApBot through POST /api/chat. The endpoint accepts JSON and keeps the chatbot model separate from presentation code. Intent and probability can be returned for evaluation and backend decisions, while technical confidence values remain hidden from the normal customer interface.

Direction	JSON contract
Request	{"message": "How do I check out?"}
Response	reply, predicted intent, probability and optional action URL
Invalid input	HTTP 400 with a controlled validation message

Frontend interaction rules

- Do not force the full chat window open when the page loads.
- Show a small, natural ApBot greeting near the launcher.
- Keep the conversation readable on desktop and mobile screens.
- Present only customer-facing language; do not show raw intent names or confidence scores.
- Use the host website only for the navigation actions required by the chatbot flow.

ApBot Testing & Acceptance

Testing approach

- Validate intents.json structure, unique tags, patterns and response lists.
- Check that preprocessing creates the expected token and Bag-of-Words representation.
- Evaluate training and validation accuracy without treating training accuracy as the only quality measure.
- Test unseen messages for every intent group, including unknown and low-confidence input.
- Verify REST validation, JSON fields, frontend display and approved website actions.

Representative acceptance cases

ID	Input	Expected ApBot result	Coverage
T01	Hello	Greeting response	Covered
T02	What products do you have?	Product-search intent	Covered
T03	Are there any offers?	Offer guidance	Covered
T04	Tell me about this product	Detailed information response	Covered
T05	Show a similar brand	Alternative-product guidance	Covered
T06	Help me choose the best option	Decision-support response	Covered
T07	Take me to checkout	Checkout guidance/action	Covered
T08	I need a representative	Contact-support response	Covered
T09	What is the weather?	Professional scope fallback	Covered
T10	Empty JSON message	HTTP 400 validation	Covered

Acceptance criteria

ApBot is acceptable when valid messages receive relevant responses, unsupported messages fail safely, the endpoint returns stable JSON, the chat UI remains usable and no technical prediction details are exposed to ordinary customers. Final pass/fail evidence should be recorded in the configured runtime environment.

Installation & SRS Deliverables

Local setup for ApBot

- Step 1: Install Python 3.12 and create the project environment with `uv venv --python 3.12 .venv`.
- Step 2: Activate the environment and install the application requirements.
- Step 3: Install notebook requirements when retraining or reviewing `Chatbot_Training.ipynb`.
- Step 4: Confirm that `intents.json`, `chatbot_model.keras`, `words.pkl` and `classes.pkl` exist in `chatbot/`.
- Step 5: Start the Flask host application with `python app.py`.
- Step 6: Open the website, launch ApBot and execute the acceptance conversations from page 17.

ApBot files

File	Purpose
<code>chatbot/intents.json</code>	Training patterns, intent tags and response groups
<code>chatbot/Chatbot_Training.ipynb</code>	Preprocessing, vectorisation, model training and evaluation
<code>chatbot/engine.py</code>	Runtime preprocessing, model inference and response selection
<code>chatbot/chatbot_model.keras</code>	Saved TensorFlow/Keras classifier
<code>chatbot/words.pkl</code>	Saved vocabulary and feature order
<code>chatbot/classes.pkl</code>	Saved output intent-class order

SRS deliverables coverage

Required deliverable	Location in project
Problem definition and design specification	This ApBot-focused document
Dialog-flow and DFD diagrams	Pages 8-10 and 15
Source code and Jupyter notebook	Repository <code>chatbot/</code> directory
Test data and test cases	<code>intents.json</code> and page 17
Installation instructions	This page and repository README
Website integration	Flask REST endpoint and frontend chat widget
Working demonstration	Run the integrated chatbot using the documented setup

ApBot Screenshot Placeholders

The spaces below are intentionally empty for final ApBot screenshots. They are limited to chatbot evidence rather than general storefront views.

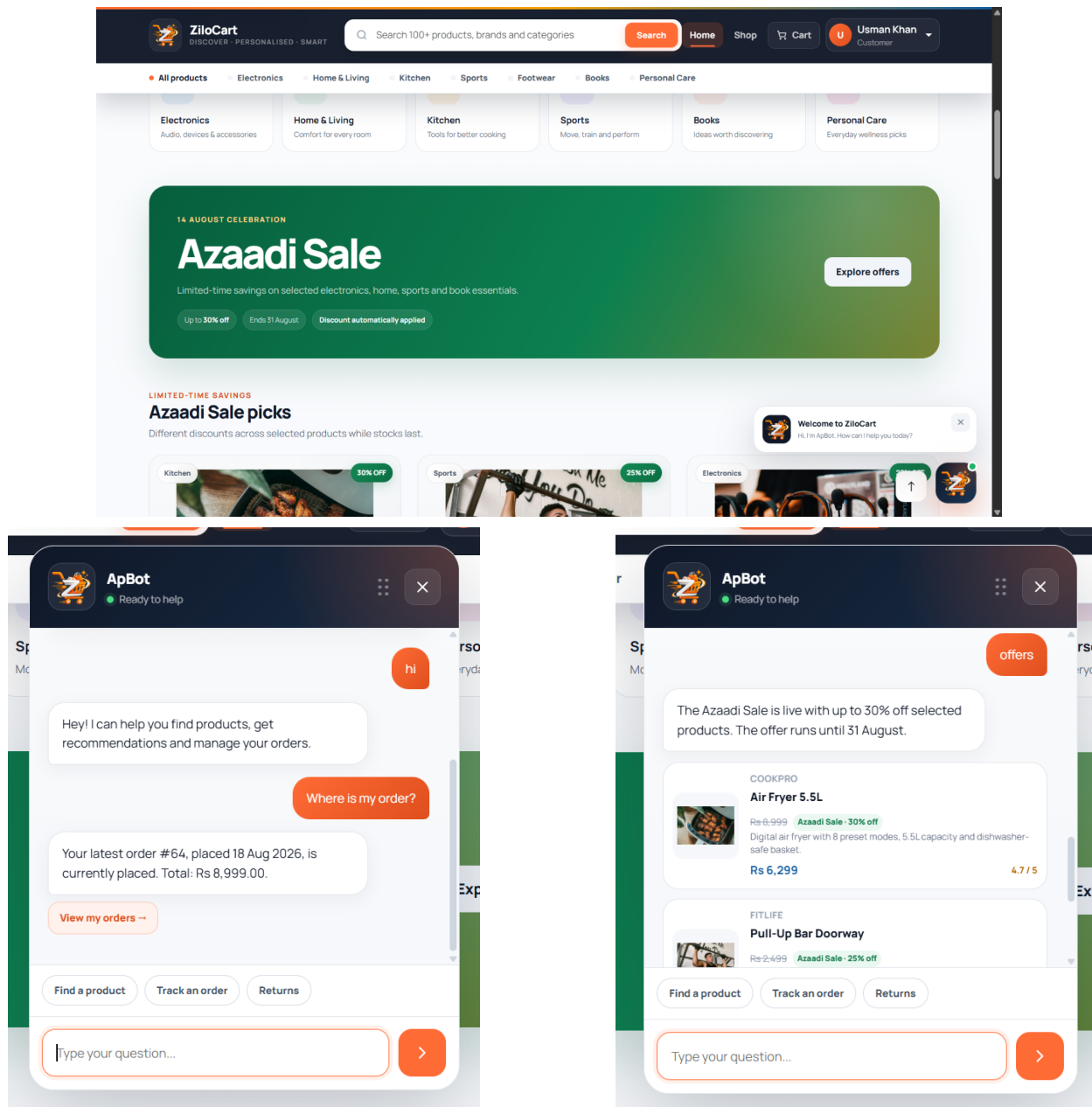


Figure 5: ApBot entry experience integrated into the host website.

Figure 6: ApBot interpreting a customer message and returning useful guidance.

Technical Evidence & Conclusion

```

File Edit View Run Kernel Tabs Settings Help
engine.py intents.json Chatbot_Training.ipynb
Python 3 (ipykernel)

confidence = float(probabilities[test_position])

return predicted_tag, confidence

[54]: test_messages = [
    "Hey, how are you?",
    "Can you suggest something useful?",
    "I need information about a product",
    "Where has my package reached?",
    "I am ready to complete my purchase",
    "How do I get my money back?",
    "Can I talk to customer service?",
    "See you next time"
]

for message in test_messages:
    tag, confidence = predict_intent(message)

    print("Message:", message)
    print("Predicted intent:", tag)
    print("Confidence:", round(confidence * 100, 2), "%")
    print("-" * 50)

Message: Hey, how are you?
Predicted intent: greeting
Confidence: 99.78 %
-----

```

ApBot satisfies the central requirements of the supplied SRS as an AI/ML e-commerce assistant that accepts ordinary human text, performs NLTK-based preprocessing, creates a Bag-of-Words vector and uses a TensorFlow/Keras model to classify the customer's intent. Its intent dataset provides supervised patterns and approved responses for product help, offers, order questions, checkout, payment, delivery, returns and support.

The solution separates the frontend chat interface, Flask backend and machine-learning classifier through a REST contract. This design keeps the chatbot understandable, testable and suitable for integration into ZiloCart without turning the ApBot report into documentation for the complete store. Its known limitations—training-data dependence, ambiguity and finite intent coverage—are handled through validation and a professional fallback response.